README

Exercise 1 Gesture Input Sketch

Real-Time Gesture-Based Regression Using Mouse Position, Mouse Movement Speed, OSC Communication, and Wekinator

`GestureInput.pde`

Contents

1 Purpose of This Processing Sketch **2**

2 Role of This Sketch in Exercise 1 **2**

3 Required Processing Libraries **2**

3.1 oscP5 . . . 3

3.2 netP5 . . . 3

4 Full Code **3**

5 Line-by-Line Explanation **5**

5.1 Header Comment . . . 5

6 Importing Libraries **6**

7 OSC Communication Variables **7**

8 Font Variable **7**

9 Feature Variables **7**

9.1 xNorm . . . 7

9.2 yNorm . . . 7

9.3 speedNorm . . . 8

10 Maximum Speed Value **8**

11 The setup() Function **8**

11.1 Window Size and Renderer . . . 8

11.2 Smooth Drawing . . . 9

11.3 Creating a Font . . . 9

11.4 Starting OSC Communication . . . 9

12 The draw() Function **10**

12.1 Clearing the Screen . . . 10

13 Feature 1 — Normalized Mouse X Position **10**

14 Feature 2 — Normalized Mouse Y Position **11**

15 Feature 3 — Mouse Movement Speed **12**

15.1 Normalizing the Speed . . . 12

16 Drawing Mouse Feedback **13**

17 Sending Data to Wekinator **13**

18 Displaying Debug Information **14**

18.1 Displaying the Current Input Values . . . 14

19 The sendOsc() Function **15**

19.1 Creating the OSC Message . . . 15

19.2 Adding the Three Input Features . . . 16

19.3 Sending the Message . . . 16

20 What the Sketch Sends to Wekinator **17**

21 Why Normalization Is Important **17**

22 Compatibility with Exercise 1 Requirements **18**

23 Expected Behavior When Running the Sketch **18**

24 Correct Wekinator Configuration for This Input Sketch **18**

25 Important Notes for Testing **19**

25.1 Check 1 — X Value . . . 19

25.2 Check 2 — Y Value . . . 19

25.3 Check 3 — Speed Value . . . 19

25.4 Check 4 — OSC Connection . . . 19

26 Why This Sketch Is Suitable for Gesture-Based Regression **19**
27 Summary **20**

1. Purpose of This Processing Sketch

This Processing sketch is the **input / data acquisition sketch** for **Lab 3 — Exercise 1: Gesture-Based Regression with Wekinator**.

The goal of this sketch is to capture a mouse gesture in real time and convert that gesture into three numerical input features:

1. **Horizontal mouse position** — normalized `x` value
2. **Vertical mouse position** — normalized `y` value
3. **Mouse movement speed** — normalized speed value

These three values are sent continuously to **Wekinator** using **OSC communication**.

The sketch does not perform machine learning by itself. Its job is to collect gesture data and send it to Wekinator, where the regression model is trained. Wekinator then learns how these input gestures should control the visual output in the second Processing sketch.

2. Role of This Sketch in Exercise 1

Exercise 1 requires a system where mouse gestures are translated into continuous visual changes. This input sketch is responsible for the first part of that system.

The complete data flow is:

```
Mouse gesture in Processing
    ↓
Extract x, y, and speed
    ↓
Normalize the values between 0 and 1
    ↓
Send the 3 values through OSC
    ↓
Wekinator receives them as 3 input features
    ↓
Wekinator predicts 2 regression outputs
    ↓
The output sketch visualizes circle size and outline transparency
```

This sketch is compatible with Exercise 1 because it provides exactly the three required input features:

```
Input 1 -> normalized x position
Input 2 -> normalized y position
Input 3 -> normalized movement speed
```

It sends them to Wekinator using:

```
OSC address:  /wek/inputs
Destination port:  6448
```

3. Required Processing Libraries

The sketch uses two Processing libraries:

```
import oscP5.*;
import netP5.*;
```

3.1. oscP5

`oscP5` is the Processing library used for sending and receiving OSC messages.

OSC means **Open Sound Control**. It is a communication protocol commonly used in creative coding, multimedia systems, audio-visual applications, and machine learning interaction systems.

In this exercise, OSC is used to send numerical features from Processing to Wekinator.

3.2. netP5

`netP5` provides networking support for OSC communication. It allows the sketch to define the destination address and port where the OSC message should be sent.

In this sketch, `netP5` is used to create the destination address:

```
dest = new NetAddress("127.0.0.1", 6448);
```

4. Full Code

```
/**
 * Lab 3 - Exercise 1
 * Gesture-Based Regression with Wekinator
 *
 * INPUT SKETCH
 * This Processing sketch captures mouse gesture data and sends
 * 3 normalized input features to Wekinator:
 *
 * Input 1: normalized horizontal position x       -> 0.0 to 1.0
 * Input 2: normalized vertical position y         -> 0.0 to 1.0
 * Input 3: normalized mouse movement speed        -> 0.0 to 1.0
 *
 * OSC message address: /wek/inputs
 * Destination port: 6448
 *
 * Wekinator setup for this exercise:
 * Inputs: 3
 * Outputs: 2
 * Model: KNN Regressor, k = 3
 */

import oscP5.*;
import netP5.*;

OscP5 oscP5;
NetAddress dest;

PFont f;

float xNorm;
float yNorm;
float speedNorm;

// This value defines what we consider a "fast" mouse movement.
```

```
// A speed of 50 pixels per frame or more becomes 1.0.
// Slower speeds are scaled between 0.0 and 1.0.
float maxSpeed = 50.0;

void setup() {
  size(640, 480, P2D);
  smooth();

  f = createFont("Courier", 16);
  textFont(f);

  // This sketch sends data to Wekinator on port 6448.
  // The local receiving port 9000 is not important here,
  // but oscP5 still needs a port to initialize.
  oscP5 = new OscP5(this, 9000);
  dest = new NetAddress("127.0.0.1", 6448);
}

void draw() {
  background(0);

  // 1. Normalize mouse x position between 0 and 1
  xNorm = mouseX / (float) width;
  xNorm = constrain(xNorm, 0, 1);

  // 2. Normalize mouse y position between 0 and 1
  yNorm = mouseY / (float) height;
  yNorm = constrain(yNorm, 0, 1);

  // 3. Calculate mouse movement speed in pixels per frame
  float speed = dist(mouseX, mouseY, pmouseX, pmouseY);

  // Normalize speed between 0 and 1
  speedNorm = speed / maxSpeed;
  speedNorm = constrain(speedNorm, 0, 1);

  // Draw a small visual feedback circle at the mouse position
  noStroke();
  fill(255);
  ellipse(mouseX, mouseY, 12, 12);

  // Send the 3 features continuously to Wekinator
  sendOsc();

  // Display values on screen for checking/debugging
  fill(255);
  text("Lab 3 - Exercise 1 Input Sketch", 10, 30);
  text("Sending 3 normalized inputs to Wekinator", 10, 55);
  text("OSC address: /wek/inputs", 10, 80);
  text("Destination port: 6448", 10, 105);

  text("x normalized      = " + nf(xNorm, 1, 3), 10, 150);
  text("y normalized      = " + nf(yNorm, 1, 3), 10, 175);
  text("speed normalized  = " + nf(speedNorm, 1, 3), 10, 200);
}

void sendOsc() {
  OscMessage msg = new OscMessage("/wek/inputs");

  // These are the exact 3 inputs required by the exercise:
  msg.add(xNorm);
  msg.add(yNorm);
  msg.add(speedNorm);
```

```
  oscP5.send(msg, dest);
}
```

5. Line-by-Line Explanation

5.1. Header Comment

```
/**
 * Lab 3 - Exercise 1
 * Gesture-Based Regression with Wekinator
```

This is a multi-line comment. It does not affect the execution of the program. It describes the purpose of the sketch.

The sketch belongs to Lab 3, Exercise 1, where the main topic is **gesture-based regression** using Wekinator.

```
 * INPUT SKETCH
```

This line identifies the sketch as the input sketch. It means this program is responsible for collecting input data and sending it to Wekinator.

```
 * This Processing sketch captures mouse gesture data and sends
 * 3 normalized input features to Wekinator:
```

This explains the main function of the sketch. It captures mouse movement and converts it into three numbers.

The word **normalized** is important. It means the values are converted to a common range, usually from `0.0` to `1.0`.

```
 * Input 1: normalized horizontal position x       -> 0.0 to 1.0
 * Input 2: normalized vertical position y         -> 0.0 to 1.0
 * Input 3: normalized mouse movement speed        -> 0.0 to 1.0
```

These lines define the three input features:

- `xNorm`: horizontal mouse position
- `yNorm`: vertical mouse position
- `speedNorm`: movement speed

All three values are scaled between `0.0` and `1.0`.

```
 * OSC message address: /wek/inputs
 * Destination port: 6448
```

These lines describe how the sketch communicates with Wekinator.

Wekinator expects input data at the OSC address:

```
/wek/inputs
```

and it usually listens for input messages on port:

```
6448
```

```
 * Wekinator setup for this exercise:
 * Inputs: 3
 * Outputs: 2
 * Model: KNN Regressor, k = 3
 */
```

These lines summarize the required Wekinator setup:

- Wekinator receives 3 inputs.
- Wekinator produces 2 outputs.
- The model type is regression.
- The algorithm is K-Nearest Neighbors with `k = 3`.

The comment ends with:

```
*/
```

6. Importing Libraries

```
import oscP5.*;
```

This line imports the `oscP5` library.

The `*` means that all useful classes from the `oscP5` library are made available to the sketch.

This library is needed because the sketch sends OSC messages to Wekinator.

```
import netP5.*;
```

This line imports the `netP5` library.

This library is used together with `oscP5` to define the destination address and port for OSC communication.

Without these imports, Processing would not recognize classes such as:

```
OscP5
NetAddress
OscMessage
```

7. OSC Communication Variables

```
OscP5 oscP5;
```

This declares a variable named `oscP5`.

Its type is `OscP5`, which means it will store the main OSC communication object.

This object is responsible for managing OSC communication in the Processing sketch.

```
NetAddress dest;
```

This declares a variable named `dest`.

Its type is `NetAddress`.

This variable stores the destination where OSC messages will be sent.

In this sketch, the destination is Wekinator running on the same computer at port `6448`.

8. Font Variable

```
PFont f;
```

This declares a font variable named `f`.

`PFont` is a Processing type used for fonts.

The sketch uses this font to display information on the Processing window, such as the normalized `x`, `y`, and speed values.

9. Feature Variables

```
float xNorm;
float yNorm;
float speedNorm;
```

These three variables store the three input features that will be sent to Wekinator.

9.1. xNorm

`xNorm` stores the normalized horizontal position of the mouse.

- When the mouse is at the far left of the window, `xNorm` is close to `0.0`.
- When the mouse is at the far right of the window, `xNorm` is close to `1.0`.

9.2. yNorm

`yNorm` stores the normalized vertical position of the mouse.

- When the mouse is at the top of the window, `yNorm` is close to `0.0`.
- When the mouse is at the bottom of the window, `yNorm` is close to `1.0`.

9.3. speedNorm

`speedNorm` stores the normalized movement speed of the mouse.

- When the mouse is not moving, `speedNorm` is close to `0.0`.
- When the mouse moves quickly, `speedNorm` increases toward `1.0`.

These variables are declared globally, so they can be used inside both `draw()` and `sendOsc()`.

10. Maximum Speed Value

```
// This value defines what we consider a "fast" mouse movement.
// A speed of 50 pixels per frame or more becomes 1.0.
// Slower speeds are scaled between 0.0 and 1.0.
float maxSpeed = 50.0;
```

The first three lines are comments explaining the role of `maxSpeed`.

```
float maxSpeed = 50.0;
```

This creates a floating-point variable named `maxSpeed` and assigns it the value `50.0`.

This value is used to normalize mouse speed.

The sketch calculates speed in pixels per frame. For example:

```
speed = 0 pixels per frame -> speedNorm = 0.0
speed = 25 pixels per frame -> speedNorm = 0.5
speed = 50 pixels per frame -> speedNorm = 1.0
speed > 50 pixels per frame -> speedNorm is limited to 1.0
```

This is useful because Wekinator works better when input values are in a consistent range.

11. The setup() Function

```
void setup() {
```

`setup()` is a special Processing function.

It runs once when the sketch starts.

It is used to configure the window, initialize libraries, create fonts, and set up OSC communication.

11.1. Window Size and Renderer

```
  size(640, 480, P2D);
```

This creates a Processing window with:

```
width = 640 pixels
height = 480 pixels
```

The third parameter, `P2D`, tells Processing to use the 2D renderer.

This is suitable because the sketch only needs simple 2D drawing: text and a small circle.

11.2. Smooth Drawing

```
smooth();
```

This enables anti-aliasing.

Anti-aliasing makes drawn shapes smoother, especially the small circle that follows the mouse.

11.3. Creating a Font

```
f = createFont("Courier", 16);
```

This creates a font object.

The font name is:

```
Courier
```

The font size is:

```
16
```

Courier is useful for debugging text because it is a monospaced font. This means each character has the same width, so the values line up clearly on screen.

```
textFont(f);
```

This tells Processing to use the font stored in `f` for all future text drawing.

11.4. Starting OSC Communication

```
// This sketch sends data to Wekinator on port 6448.
// The local receiving port 9000 is not important here,
// but oscP5 still needs a port to initialize.
oscP5 = new OscP5(this, 9000);
```

The comments explain that the important destination port is `6448`, because that is where Wekinator receives input.

```
oscP5 = new OscP5(this, 9000);
```

This creates the OSC communication object.

The value `9000` is the local port used by this Processing sketch. In this sketch, it is not the main focus because the sketch mainly sends data rather than receiving it.

The important point is that this line initializes the OSC system so the sketch can send messages.

```
dest = new NetAddress("127.0.0.1", 6448);
```

This creates the destination address for OSC messages.

```
127.0.0.1
```

means the local computer. It is also called `localhost`.

```
6448
```

is the port where Wekinator receives input messages.

So this line means:

```
Send OSC messages to Wekinator on the same computer, using port 6448.
```

```
}
```

This closes the `setup()` function.

12. The draw() Function

```
void draw() {
```

`draw()` is another special Processing function.

Unlike `setup()`, which runs once, `draw()` runs continuously in a loop.

By default, Processing calls `draw()` around 60 times per second.

This is important because the sketch needs to send real-time gesture data continuously to Wekinator.

12.1. Clearing the Screen

```
  background(0);
```

This paints the entire window black at the beginning of every frame.

The value `0` means black in grayscale.

This prevents old drawings from remaining on the screen.

13. Feature 1 — Normalized Mouse X Position

```
  // 1. Normalize mouse x position between 0 and 1
  xNorm = mouseX / (float) width;
```

The comment explains that this part calculates the normalized horizontal mouse position.

`mouseX` is a built-in Processing variable. It stores the current horizontal position of the mouse in pixels.

`width` is a built-in Processing variable. It stores the width of the sketch window, which is `640` in this program.

`(float) width` converts `width` into a floating-point number.

This is important because the result should be a decimal value such as `0.250`, `0.500`, or `0.875`, not just an integer.

The formula is:

```
xNorm = mouseX / width
```

Examples:

```
mouseX = 0 -> xNorm = 0.0
mouseX = 320 -> xNorm = 0.5
mouseX = 640 -> xNorm = 1.0
```

This converts the mouse x position from a pixel range into a normalized range.

```
xNorm = constrain(xNorm, 0, 1);
```

This line makes sure `xNorm` never goes below `0` and never goes above `1`.

The function:

```
constrain(value, minimum, maximum)
```

limits a value inside a specific range.

This is useful because the mouse can sometimes briefly produce values outside the visible window area.

14. Feature 2 — Normalized Mouse Y Position

```
// 2. Normalize mouse y position between 0 and 1
yNorm = mouseY / (float) height;
```

The comment explains that this part calculates the normalized vertical mouse position.

`mouseY` is a built-in Processing variable. It stores the current vertical position of the mouse in pixels.

`height` is a built-in Processing variable. It stores the height of the sketch window, which is `480` in this program.

The formula is:

```
yNorm = mouseY / height
```

Examples:

```
mouseY = 0 -> yNorm = 0.0
mouseY = 240 -> yNorm = 0.5
mouseY = 480 -> yNorm = 1.0
```

In Processing, the top of the window has `y = 0`, and the bottom of the window has the largest y value.

So:

```
Top of window -> yNorm close to 0.0
Bottom of window -> yNorm close to 1.0
```

```
yNorm = constrain(yNorm, 0, 1);
```

This makes sure `yNorm` stays inside the range from `0` to `1`.

15. Feature 3 — Mouse Movement Speed

```
// 3. Calculate mouse movement speed in pixels per frame
float speed = dist(mouseX, mouseY, pmouseX, pmouseY);
```

This line calculates the movement speed of the mouse.

`float speed` creates a local variable named `speed`.

It is local because it is created inside `draw()` and only used inside `draw()`.

`dist(mouseX, mouseY, pmouseX, pmouseY)` calculates the distance between two points:

```
Current mouse position:  mouseX, mouseY
Previous mouse position:  pmouseX, pmouseY
```

`pmouseX` and `pmouseY` are built-in Processing variables. They store the mouse position from the previous frame.

So the speed is calculated as the distance between the current mouse position and the previous mouse position.

If the mouse barely moved, the distance is small.

If the mouse moved quickly, the distance is large.

This is not speed in meters per second. It is a practical creative-coding speed measurement:

```
pixels moved per frame
```

That is enough for this exercise because Wekinator only needs a useful numerical feature describing how fast the gesture is.

15.1. Normalizing the Speed

```
// Normalize speed between 0 and 1
speedNorm = speed / maxSpeed;
```

This converts the raw speed value into a normalized value.

The formula is:

```
speedNorm = speed / maxSpeed
```

Since `maxSpeed` is `50.0`, the calculation means:

```
speedNorm = speed / 50.0
```

Examples:

```
speed = 0 -> speedNorm = 0.0
speed = 10 -> speedNorm = 0.2
speed = 25 -> speedNorm = 0.5
speed = 50 -> speedNorm = 1.0
speed = 80 -> speedNorm = 1.6 before constraining
```

```
speedNorm = constrain(speedNorm, 0, 1);
```

This limits the normalized speed between `0` and `1`.

So if the raw calculation gives a value greater than `1`, it is clipped to `1`.

For example:

```
speed = 80
speed / 50 = 1.6
constrain(1.6, 0, 1) = 1.0
```

This is important because Exercise 1 expects normalized input values.

16. Drawing Mouse Feedback

```
// Draw a small visual feedback circle at the mouse position
noStroke();
```

This disables the outline around shapes.

The small feedback circle will have no border.

```
fill(255);
```

This sets the fill color to white.

In grayscale Processing color:

```
0 = black
255 = white
```

```
ellipse(mouseX, mouseY, 12, 12);
```

This draws a small circle at the current mouse position.

The parameters are:

```
ellipse(x, y, width, height)
```

So this line means:

```
Draw a 12 by 12 pixel white circle at the current mouse position.
```

This visual feedback helps the user see where the mouse gesture is happening.

This circle is not sent to Wekinator. It is only for the user interface of the input sketch.

17. Sending Data to Wekinator

```
// Send the 3 features continuously to Wekinator
sendOsc();
```

This calls the custom function `sendOsc()`.

Because this line is inside `draw()`, it is executed every frame.

That means the sketch continuously sends the latest values of:

```
xNorm
yNorm
speedNorm
```

to Wekinator.

This is necessary because Wekinator needs real-time input while recording training examples and while running the trained model.

18. Displaying Debug Information

```
// Display values on screen for checking/debugging
fill(255);
```

This sets the text color to white.

The debugging text appears on the black background.

```
text("Lab 3 - Exercise 1 Input Sketch", 10, 30);
```

This displays the title of the sketch.

The text starts at pixel position:

```
x = 10
y = 30
```

```
text("Sending 3 normalized inputs to Wekinator", 10, 55);
```

This explains what the sketch is doing.

```
text("OSC address: /wek/inputs", 10, 80);
```

This displays the OSC address used to send data.

This is useful because Wekinator must be configured to receive messages from:

```
/wek/inputs
```

```
text("Destination port: 6448", 10, 105);
```

This displays the destination port.

Wekinator must listen on port:

```
6448
```

18.1. Displaying the Current Input Values

```
text("x normalized      = " + nf(xNorm, 1, 3), 10, 150);
```

This displays the current normalized x value.

`nf(xNorm, 1, 3)` formats the number so it is easier to read.

The `3` means it shows three digits after the decimal point.

Example:

```
0.842
```

```
  text("y normalized      = " + nf(yNorm, 1, 3), 10, 175);
```

This displays the current normalized y value.

```
  text("speed normalized  = " + nf(speedNorm, 1, 3), 10, 200);
```

This displays the current normalized speed value.

These displayed values are important for checking whether the sketch is working correctly.

For example:

- Moving the mouse left and right should change `x normalized`.
- Moving the mouse up and down should change `y normalized`.
- Moving the mouse faster should increase `speed normalized`.
- Keeping the mouse still should make `speed normalized` close to `0.000`.

```
}
```

This closes the `draw()` function.

19. The sendOsc() Function

```
void sendOsc() {
```

This defines a custom function named `sendOsc()`.

The purpose of this function is to build an OSC message and send it to Wekinator.

It keeps the OSC sending code organized instead of putting all the message creation directly inside `draw()`.

19.1. Creating the OSC Message

```
  OscMessage msg = new OscMessage("/wek/inputs");
```

This creates a new OSC message.

The message address is:

```
/wek/inputs
```

This is the standard input message address expected by Wekinator.

Wekinator uses this address to understand that the received numbers are input features.

19.2. Adding the Three Input Features

```
// These are the exact 3 inputs required by the exercise:
msg.add(xNorm);
```

The comment explains that the following three values are the required inputs for Exercise 1.

```
msg.add(xNorm);
```

This adds the first input value to the OSC message.

This is the normalized horizontal mouse position.

In Wekinator, this becomes:

```
Input 1
```

```
msg.add(yNorm);
```

This adds the second input value to the OSC message.

This is the normalized vertical mouse position.

In Wekinator, this becomes:

```
Input 2
```

```
msg.add(speedNorm);
```

This adds the third input value to the OSC message.

This is the normalized mouse movement speed.

In Wekinator, this becomes:

```
Input 3
```

The order is very important.

Wekinator receives the values in exactly this order:

```
1.  xNorm
2.  yNorm
3.  speedNorm
```

So the Wekinator project should be configured with **3 inputs**.

19.3. Sending the Message

```
oscP5.send(msg, dest);
```

This sends the OSC message to the destination address stored in `dest`.

Earlier in `setup()`, `dest` was defined as:

```
dest = new NetAddress("127.0.0.1", 6448);
```

So this line sends the message to:

```
IP address:  127.0.0.1
Port:  6448
```

This is exactly where Wekinator should be listening for input data.

```
}
```

This closes the `sendOsc()` function.

20. What the Sketch Sends to Wekinator

Every frame, the sketch sends one OSC message with this structure:

```
Address:  /wek/inputs
Arguments:
   1.  xNorm
   2.  yNorm
   3.  speedNorm
```

Example OSC message:

```
/wek/inputs 0.742 0.516 0.284
```

This means:

```
Mouse is around 74.2% across the width of the window.
Mouse is around 51.6% down the height of the window.
Mouse is moving at around 28.4% of the chosen maximum speed.
```

21. Why Normalization Is Important

Normalization means converting values into a standard range.

In this sketch, all input values are converted to the range:

```
0.0 to 1.0
```

This is useful for Wekinator because the three features originally have different scales:

```
mouseX: 0 to 640
mouseY: 0 to 480
speed:  can vary depending on movement
```

Without normalization, one feature may dominate the learning process simply because its numerical range is larger.

After normalization:

```
xNorm:  0.0 to 1.0
yNorm:  0.0 to 1.0
speedNorm:  0.0 to 1.0
```

This makes the input features more balanced and suitable for regression.

22. Compatibility with Exercise 1 Requirements

This sketch satisfies the input requirements of Exercise 1 in the following way:

Exercise 1 Requirement	How This Sketch Satisfies It
Capture mouse gesture data	Uses `mouseX`, `mouseY`, `pmouseX`, and `pmouseY`
Extract horizontal position	Calculates `xNorm`
Extract vertical position	Calculates `yNorm`
Extract movement speed	Uses `dist(mouseX, mouseY, pmouseX, pmouseY)`
Normalize values between 0 and 1	Uses division by width, height, and `maxSpeed`, then `constrain()`
Send values to Wekinator	Uses `oscP5.send(msg, dest)`
Use OSC address `/wek/inputs`	Creates `OscMessage("/wek/inputs")`
Send to Wekinator input port	Sends to `127.0.0.1`, port `6448`
Provide 3 inputs	Adds `xNorm`, `yNorm`, and `speedNorm` to the OSC message
Support regression training	Sends continuous numerical values, suitable for a regression model

23. Expected Behavior When Running the Sketch

When the sketch runs, a black window appears.

A small white circle follows the mouse position.

The screen displays:

```
Lab 3 - Exercise 1 Input Sketch
Sending 3 normalized inputs to Wekinator
OSC address:  /wek/inputs
Destination port:  6448

x normalized = ...
y normalized = ...
speed normalized = ...
```

When the mouse moves horizontally, the `x normalized` value changes.

When the mouse moves vertically, the `y normalized` value changes.

When the mouse moves quickly, the `speed normalized` value increases.

When the mouse stops, the `speed normalized` value becomes close to `0.000`.

This behavior confirms that the sketch is extracting the correct gesture features.

24. Correct Wekinator Configuration for This Input Sketch

In Wekinator, the project should be configured like this:

```
Input port:  6448
Input message:  /wek/inputs
Number of inputs:  3
Number of outputs:  2
```

```
Model type:  Regression
Algorithm:  KNN Regressor
k:  3
```

The three Wekinator inputs should correspond to:

```
Input 1:  x position
Input 2:  y position
Input 3:  speed
```

The two Wekinator outputs should later control:

```
Output 1:  circle size
Output 2:  outline alpha / transparency
```

25. Important Notes for Testing

Before training Wekinator, run this Processing input sketch and check that the values on screen are changing correctly.

25.1. Check 1 — X Value

Move the mouse left and right.

Expected result:

```
x normalized changes between 0.0 and 1.0
```

25.2. Check 2 — Y Value

Move the mouse up and down.

Expected result:

```
y normalized changes between 0.0 and 1.0
```

25.3. Check 3 — Speed Value

Move the mouse slowly, then quickly.

Expected result:

```
slow movement -> lower speedNorm
fast movement -> higher speedNorm
no movement -> speedNorm close to 0.000
```

25.4. Check 4 — OSC Connection

Open Wekinator and make sure the input port is set to `6448`.

When Wekinator receives the input successfully, its input sliders or input values should change as the mouse moves.

26. Why This Sketch Is Suitable for Gesture-Based Regression

This sketch is suitable for gesture-based regression because it does not produce discrete labels such as:

```
left gesture
right gesture
fast gesture
slow gesture
```

Instead, it produces continuous values:

```
xNorm
yNorm
speedNorm
```

These continuous values describe the gesture numerically.

That is exactly what regression needs. Regression learns a mapping from continuous input features to continuous output values.

In this exercise, Wekinator learns a mapping like:

```
x position + y position + speed
    ->
circle size + outline transparency
```

This makes the system expressive because small changes in the mouse movement can create smooth changes in the visual output.

27. Summary

This Processing sketch is the input component of the Exercise 1 gesture-regression system.

It performs four main tasks:

1. Captures real-time mouse data.
2. Extracts three gesture features: `x`, `y`, and speed.
3. Normalizes all features between `0.0` and `1.0`.
4. Sends the three values to Wekinator using OSC.

The sketch is fully compatible with the Exercise 1 input requirements because it sends exactly three normalized input features to Wekinator through `/wek/inputs` on port `6448`.

It should be used together with a Wekinator regression project and the Exercise 1 output visualization sketch.